MunyaMap - Officializing The Namibian Land Claim Process

Hack Without Borders

15 March 2026

Anren Guo, Giovanni Murzi, Sebastian Southworth, Tristan Dermody, Yuta Yamanaka

1.0 Introduction

In the Osumati region of Namibia (Figure 1), there exists problems with the land distribution process. For instance, information often slips from the paper database [1]. The current system fails to prevent illegal land acquisitions and expansions by the upper-class, leaving the lower-class stranded with no land and exacerbating the socioeconomic divide [1]. The team aims to address this issue by designing a website for civilians and officials to initiate and officialize land claims.

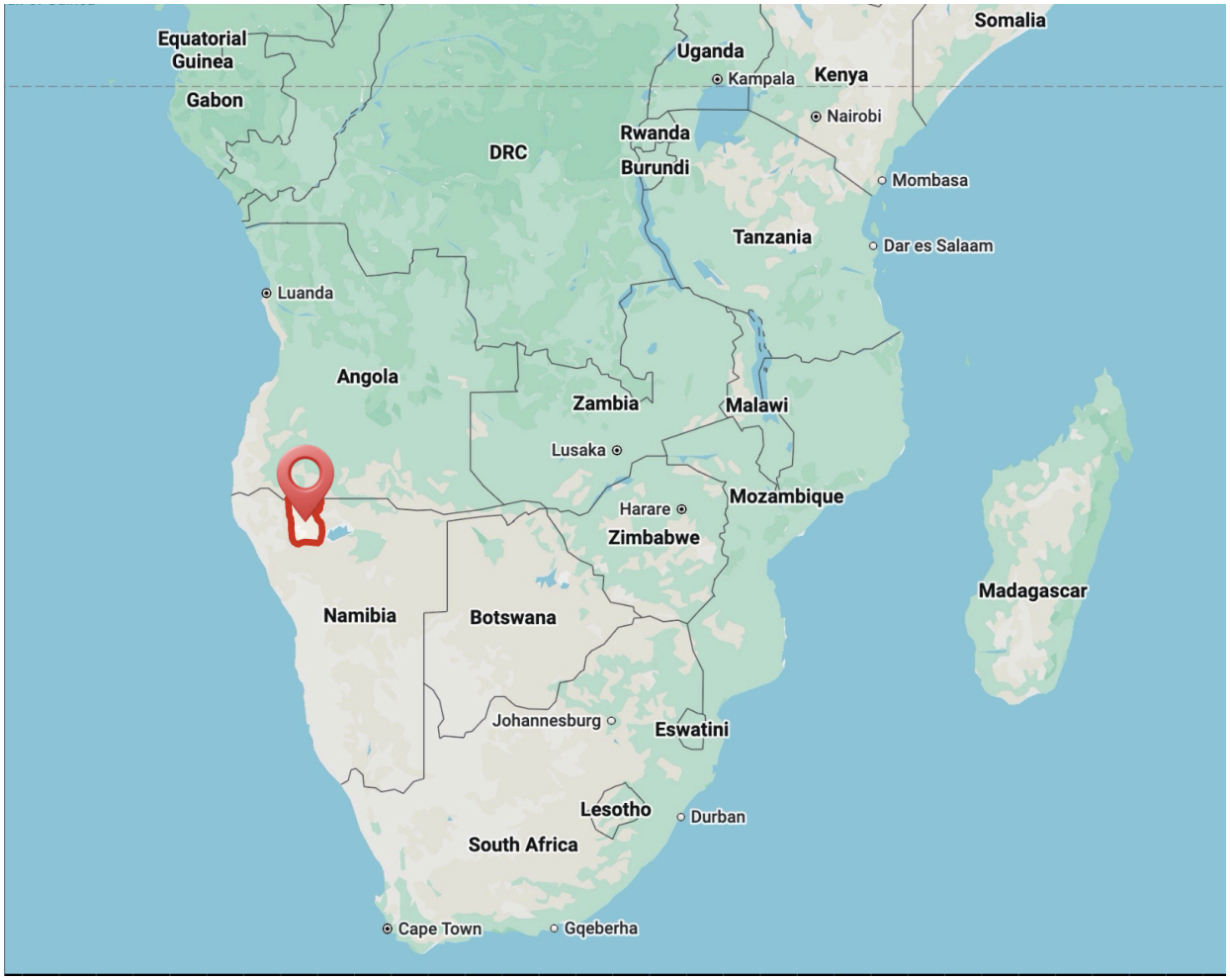

Figure 1. Osumati region in Namibia. Taken from [2].

2.0 Problem Statement

Communal land is defined as land owned by the government and held in trust for the traditional communities residing there. The term implies allowance for customary land rights, practice of customary land use (e.g. residential and rural development). These rights are fundamental to No. 5 the Communal Land Reform Act 2002 (CLRA) [1].

As per regulation 26 of the act, residents are allowed to erect fences around their property, and when done so in accordance with the act, this action is legal. However – section 44 of the act prohibits fencing on communal land without a land right certificate.

Since the 1970s, illegal fencing has been common practice. Local businesses began seeking and obtaining approval from local chiefs to fence major sections of the communal land. This trend accelerated through the 1980s, and by the early 1990s, an estimated 25-50% of the communal land had been fenced off into private property for these businesses. The local community were effectively priced out of the land, increasing poverty and the socioeconomic divide. A 2011 evaluation conducted by an Ongandjera traditional authority confirmed that fencing had only increased in scale and intensity. [1]

Existing administrative and legislative structures are insufficient. The current paper-based system is frequently "not registered in the village books and not reported at the TA's [traditional authority] head office" [1]. The reliance on paper databases, corruption and the poor recording mechanism for illegal fences fails to punish them. Additionally, many fences appear with no documentation or owner and are likely to have been constructed by individuals of higher economic class to stake their claim on the land for possible future use. [1]

The effect of these actions is profoundly negative on the local community. Due to the region-wide gentrification and corruption, the local community has been priced out of their communal land by the wealthy (who often have little cultural ties to the region). This reveals the gap: the lack of a means to properly document and officialize land ownership, and verify claims to prevent the wide-spread illegal fencing.

Thus, the need is a design solution that provides an organized and structured means of claiming and officializing land.

3.0 Interest Holders

The interest holders in this project are listed in Table 1 below.

Table 1. Interest Holder list.

Interest Holder	Interest	Impact
Traditional Authority (TA)	Maintain tribal sovereignty and stewardship .	The traditional authorities will be one of three intended users, and thus, their perspective will impact design decisions.

Communal Land Board (CLB)	The communal land board will be the final authority on whether claims pass. Their job is to officialize land claims, and the process is made more rigorous by a solution.	The communicable land board will be another user, and their perspective will impact design decisions.

4.0 Land Claim Processes and Constraints

In Namibia, to litigate a land claim, there is a formal 3-step process [3]:

1. Apply to the Traditional Authority, who reviews the request and forwards it to the Communal Land Board if approved
2. The Communal Land Board checks existing fences, ensures the land does not overlap with protected communal resources (e.g. water), and ensures compliance with area limits
    a. up to 20 hectares for residential land
    b. up to 50 hectares for farmland (unless special approval is given).
3. If approved, the right is recorded and the applicant will be granted a certificate

In the case of disputes, such as overlapping claims, unregistered occupation, or conflicts between individual and communal rights, the issue also follows the process above [3]:

1. Traditional authorities review the disputes. If they believe it is valid, they will forward it to the Communal Land Board.
2. The Communal Land Board will double-check the dispute. If they also believe it to be valid, then court proceedings will ensue.

In the case of conflicting claims, the authorities tend to prioritize the earliest valid registered right. Finally, existing allocations that exceed limits or impact communal resources may be considered for reduction, cancellation, or reassignment [4].

5.0 Solution

MunyaMap is a web-based land claim platform designed to digitize and formalize communal land allocation in the Osumati region of Namibia. The design enables citizens, TAs and the CLB to interact with each other through a transparent and centralized process for verifying land claims. By digitizing the process, the design aims to deter fraudulent land claims by keeping permanent and up-to-date records of all existing and proposed claims.

Using an interactive satellite-generated map, civilians can outline the geographic boundaries of the land they wish to claim. The software automatically calculates the area and location of the outline and validates it against regulations and communal resource locations*. After the

civilian's submission, the claim is first reviewed by the Traditional Authority. The TA ensures that the land is not already occupied or allocated and that the claim boundaries seem reasonable and legitimate. If the TA approves the request, the claim is then digitally forwarded to the CLB. The CLB checks if the land complies with land regulations, verifies the calculated area does not surpass 50 hectares and ensures that the land does not interfere with shared communal resources. If the CLB verifies the claim, they can then send the citizen a digitally recorded land right certificate. All approvals are permanently stored digitally in the system to prevent data loss.

* The design prototype only provides functionality for farmland claims (limit of 50 hectares). Further enhancement with a separate mode for residential claims (with a different limit of 20 hectares [3]) would be a next step if deployment is to be considered.

A working prototype has been developed and can be accessed via the following GitHub page: MunyaMap

Reference List:

[1] - Google Maps, "Google Maps," *Google Maps*, 2021. [Online]. Available: https://www.google.com/maps/place/Omusati+Region (accessed Mar. 15, 2026).

[2] - Republic of Namibia, "Manual for Communal Land Boards Powers and Procedures for Handling Land Issues," 2007. Accessed: Mar. 15, 2026. [Online]. Available: https://www.lac.org.na/projects/lead/Pdf/clbTM12.pdf

[3] - R.-M. Kashululu and P. Hebinck, "Chapter 8 • The fencing question in Namibia: A case study in Omusati Region • 163 8 The fencing question in Namibia: A case study in Omusati Region." Accessed: Mar. 15, 2026. [Online]. Available: https://www.lac.org.na/projects/lead/Pdf/neither-8.pdf

[4] - "Communal Land Reform Amendment Act 11 of 2005 (GG 3550) came into force on date of publication: 8 December 2005." Available: https://www.lac.org.na/laws/annoSTAT/Communal%20Land%20Reform%20Act%205%20of%202002.pdf

Appendix A: AI Usage

AI was utilized in the document to facilitate research. The text was written by hand based on research conducted with and without AI.

Manuscript Prompt List (pasted from Gemini chat logs):

- “write me a detailed description of the dead capital problem in Zambezi region”
- “https://www.lac.org.na/projects/lead/Pdf/neither-8.pdf summarize the institutional void aspect of this report”
- “also explained the ghost fences and paper based current system”

AI usage for code (claude code):

Code + prompt:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Navibian Land Claim Portal - Omusati Region</title>



    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/leaflet.draw/1.0.4/leaflet.draw.css"
/>



    <style>

        :root { --primary: #2c3e50; --secondary: #3498db; --success: #27ae60; --warning: #f39c12; --danger:
#e74c3c; }

        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0;
background: #ecf0f1; color: #333; }



        /* Login Screen */

        #login-screen { display: flex; flex-direction: column; align-items: center; justify-content: center;
height: 100vh; }
```

```
.login-box { background: white; padding: 40px; border-radius: 8px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); width: 350px; text-align: center; }

.login-box input, .login-box select { width: 100%; padding: 10px; margin: 10px 0; border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; }

.btn { padding: 10px 15px; border: none; border-radius: 4px; background: var(--secondary); color: white; font-weight: bold; cursor: pointer; width: 100%; margin-top: 10px; }

.btn:hover { background: #2980b9; }


/* Main App Layout */

#app-screen { display: none; height: 100vh; flex-direction: column; }

header { background: var(--primary); color: white; padding: 15px 20px; display: flex; justify-content: space-between; align-items: center; }

header a { color: #a4cbed; text-decoration: none; font-size: 0.9em; }

header a:hover { text-decoration: underline; }


.main-container { display: flex; flex: 1; overflow: hidden; }

.sidebar { width: 350px; background: white; padding: 20px; overflow-y: auto; box-shadow: 2px 0 5px rgba(0,0,0,0.05); z-index: 2; }

#map { flex: 1; z-index: 1; }


/* Claim Cards */

.claim-card { border: 1px solid #ddd; padding: 15px; margin-bottom: 15px; border-radius: 6px; background: #f9f9f9; }

.claim-card h3 { margin: 0 0 10px 0; font-size: 16px; }

.claim-card p { margin: 5px 0; font-size: 14px; }

.status-badge { display: inline-block; padding: 3px 8px; border-radius: 12px; font-size: 12px; color: white; font-weight: bold;}

.status-pending { background: var(--warning); }
```

```
        .status-approved { background: var(--success); }


        /* Admin Popup Form */

        .admin-action-form { margin-top: 10px; border-top: 1px solid #ccc; padding-top: 10px; }

        .admin-action-form input { width: 100%; padding: 8px; margin-bottom: 10px; box-sizing:
border-box; }

        .btn-approve { background: var(--success); width: 48%; }

        .btn-reject { background: var(--danger); width: 48%; }

        .btn-logout { background: #7f8c8d; width: auto; padding: 8px 15px;}

    </style>

</head>

<body>


    <div id="login-screen">

        <div class="login-box">

            <h2>Portal Login</h2>

            <select id="role-select">

                <option value="citizen">Citizen of Namibia</option>

                <option value="ta">Traditional Authority (TA) Admin</option>

                <option value="clb">Communal Land Board (CLB) Admin</option>

            </select>

            <input type="text" id="email-input" placeholder="Email / Username"
value="citizen@omusati.na">

            <input type="password" id="password-input" placeholder="Password" value="password123">

            <button class="btn" onclick="login()">Access System</button>
```

```
    </div>
  </div>


  <div id="app-screen">
    <header>
      <div>
        <h2 id="header-title" style="margin: 0;">Namibian Land Claim Portal</h2>
        <span id="user-info" style="font-size: 0.9em; color: #bdc3c7;"></span>
      </div>
      <div>
        <a
href="https://www.lac.org.na/laws/annoSTAT/Communal%20Land%20Reform%20Act%205%20of%202
002.pdf" target="_blank">View Namibian Land Claim Guidelines</a>
        <button class="btn btn-logout" style="margin-left: 15px;" onclick="logout()">Logout</button>
      </div>
    </header>


    <div class="main-container">
      <div class="sidebar">
        <h3 id="sidebar-title">My Claims</h3>
        <div id="claims-list">
          </div>
      </div>


      <div id="map"></div>
```

```
    </div>
</div>

<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/leaflet.draw/1.0.4/leaflet.draw.js"></script>
<script src="https://cdn.jsdelivr.net/npm/pouchdb@8.0.1/dist/pouchdb.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@turf/turf@6/turf.min.js"></script>

<script>
    // --- SYSTEM STATE & DB ---
    const db = new PouchDB('omusati_land_registry');
    let currentUser = null; // { role, email }

    let map, drawControl;
    const allMapLayers = new L.FeatureGroup();
    let savedFeatures = []; // Pure math geometry for Turf.js collision detection

    // Simulated Protected Water (Olushandja Dam area in Omusati)
    const waterFeature = turf.polygon([[
        [14.65, -17.40], [14.70, -17.40], [14.70, -17.48], [14.65, -17.48], [14.65, -17.40]
    ]]);

    // --- 1. AUTHENTICATION ---
    function login() {
```

```
    const role = document.getElementById('role-select').value;
    const email = document.getElementById('email-input').value;

    if (!email) return alert("Please enter an email/username.");

    currentUser = { role, email };

    document.getElementById('login-screen').style.display = 'none';
    document.getElementById('app-screen').style.display = 'flex';

    // Adjust UI based on role
    document.getElementById('header-title').innerText = role === 'citizen' ? "MunyaMap Land Portal" : (role === 'ta' ? "MunyaMap: Traditional Authority Admin Portal" : "Munya Map: Communal Land Board Admin Portal");
    document.getElementById('sidebar-title').innerText = role === 'citizen' ? "My Pending Claims" : "All System Claims";
    if (role === 'citizen') document.getElementById('user-info').innerText = `Logged in as: ${email}`;

    initMap();
    loadClaims();
  }

  function logout() {
    currentUser = null;
    document.getElementById('app-screen').style.display = 'none';
    document.getElementById('login-screen').style.display = 'flex';
```

```
    if(map) { map.remove(); map = null; }
}

// --- 2. MAP INITIALIZATION ---
function initMap() {
    // Centered on Omusati Region
    map = L.map('map').setView([-17.45, 14.68], 12);
    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', { maxZoom: 18 }).addTo(map);
    map.addLayer(allMapLayers);

    // Setup Drawing Tools ONLY for Citizens
    if (currentUser.role === 'citizen') {
        drawControl = new L.Control.Draw({
            edit: false,
            draw: { marker: false, circle: false, circlemarker: false, polyline: false, polygon: true }
        });
        map.addControl(drawControl);

        // Handle Citizen Drawing
        map.on(L.Draw.Event.CREATED, async function (event) {
            const newGeoJSON = event.layer.toGeoJSON();

            // Validation 1: Size Limit (< 50 Hectares)
            const areaSqMeters = turf.area(newGeoJSON);
```

```
            const areaHectares = areaSqMeters / 10000;

            if (areaHectares > 50) {

                return alert(`❌ Claim Denied: Your plot is ${areaHectares.toFixed(2)} ha. Maximum
allowed by CLRA is 50 ha.`);

            }



            // Validation 3: Existing Claim Overlap

            let collision = savedFeatures.some(f => turf.booleanIntersects(newGeoJSON, f));

            if (collision) {

                return alert("❌ Claim Denied: Boundary overlaps with an existing pending or approved
claim.");

            }



            // If valid, save to database

            createNewClaim(newGeoJSON.geometry, areaHectares);

        });

    }

}



// --- 3. CLAIM CREATION & SYNC ---

async function createNewClaim(geometry, hectares) {

    // Generate the secure key for the TA/CLB to verify

    const cryptoKey = Math.random().toString(36).substring(2, 10).toUpperCase();



    const claim = {
```

```
        _id: 'claim_' + new Date().toISOString(),
        claimerEmail: currentUser.email,
        date: new Date().toLocaleDateString(),
        area: hectares.toFixed(2),
        boundary: geometry,
        cryptoKey: cryptoKey, // Passed to DB for admin verification
        status: 'pending_ta', // Workflow: pending_ta -> pending_clb -> approved
        type: 'land_claim'
    };

    await db.put(claim);
    alert("✅ Claim Submitted! Awaiting Traditional Authority (TA) approval.");
    loadClaims();
}

// --- 4. RENDER CLAIMS (MAP & LIST) ---
async function loadClaims() {
    allMapLayers.clearLayers();
    savedFeatures = [];
    const claimsListDiv = document.getElementById('claims-list');
    claimsListDiv.innerHTML = '';

    const result = await db.allDocs({include_docs: true});
```

```
result.rows.forEach(row => {
    const claim = row.doc;
    if(claim.type !== 'land_claim') return;

    // Filter citizen view to only show their claims
    if (currentUser.role === 'citizen' && claim.claimerEmail !== currentUser.email) return;

    // 1. Add to Math Ledger for collision detection
    savedFeatures.push(turf.feature(claim.boundary));

    // 2. Add to Map
    let color = claim.status === 'approved' ? '#27ae60' : '#f39c12';
    const layer = L.geoJSON(claim.boundary, { style: { color: color, weight: 2, fillOpacity: 0.4 } });

    // Construct Popup
    let popupContent = `
        <b>Claim ID:</b> ${claim._id.split('_')[1]}<br>
        <b>Claimer:</b> ${claim.claimerEmail}<br>
        <b>Area:</b> ${claim.area} hectares<br>
        <b>Status:</b> ${claim.status.replace('_', ' ').toUpperCase()}<br>
    `;

    // Admin Verification Form in Popup
    if ((currentUser.role === 'ta' && claim.status === 'pending_ta') ||
```

```
            (currentUser.role === 'clb' && claim.status === 'pending_clb')) {
            popupContent += `
                <div class="admin-action-form">
                    <input type="text" id="key-${claim._id}" placeholder="Enter Crypto Key to Verify">
                    <button class="btn btn-approve" onclick="verifyAndApprove('${claim._id}',
'${currentUser.role}')">Approve</button>
                    <button class="btn btn-reject" onclick="rejectClaim('${claim._id}')">Deny</button>
                </div>
            `;
        }

        layer.bindPopup(popupContent);
        allMapLayers.addLayer(layer);

        // 3. Add to Sidebar List
        let badgeClass = claim.status === 'approved' ? 'status-approved' : 'status-pending';
        let adminKeyNote = (currentUser.role !== 'citizen') ? `<br><small style="color:red;">Secure
Key: ${claim.cryptoKey}</small>` : '';

        claimsListDiv.innerHTML += `
            <div class="claim-card">
                <h3>Claim: ${claim._id.split('_')[1].substring(0, 10)}...</h3>
                <p><b>Date:</b> ${claim.date}</p>
                <p><b>Size:</b> ${claim.area} ha</p>
                <p><span class="status-badge ${badgeClass}">${claim.status.replace('_', '
').toUpperCase()}</span></p>
```

```
            ${adminKeyNote}
        </div>
    `;
  });
}

// --- 5. ADMIN VERIFICATION LOGIC ---
async function verifyAndApprove(claimId, role) {
  const enteredKey = document.getElementById(`key-${claimId}`).value;
  const claim = await db.get(claimId);

  if (enteredKey !== claim.cryptoKey) {
    return alert("❌ Invalid Crypto Key. Verification failed.");
  }

  // Advance workflow
  if (role === 'ta') claim.status = 'pending_clb';
  if (role === 'clb') claim.status = 'approved';

  await db.put(claim);
  map.closePopup();
  alert(`✅ Claim ${role.toUpperCase()} Approved.`);
  loadClaims();
}
```

```
    async function rejectClaim(claimId) {
        if(!confirm("Are you sure you want to completely deny and remove this claim?")) return;
        const claim = await db.get(claimId);
        await db.remove(claim); // If rejected, it is removed from the map
        map.closePopup();
        alert("❌ Claim has been denied and removed from the ledger.");
        loadClaims();
    }

    // Live Sync Listener (Updates UI instantly if another device makes a change)
    db.changes({ since: 'now', live: true, include_docs: true }).on('change', () => {
        if(currentUser) loadClaims();
    });

  </script>
</body>
</html>
```

“Bugs:

- Land claims from other users do not appear on citizen portal
- Every claim is submitted twice
- Startup includes an unwanted blue square
- The CLB and TA should have different keys
- Users can deny without a crypto key
- Users should not be able to lay claims on water
- Used the wrong password and it still allows access
- Claims should not be able to be able to be made outside namibia
- Claims that get rejected should receive notification for the citizen
- Additional features to be implemented:

- Use satellite image for map
- If claims overlap, give citizen ability to request a hearing
- Claims rejected by CLB should prompt CLB for a reason that's sent in the notification to the citizen
- If you click on a claim, it should move to it on the map
- add a feature where a citizen can remove a land claim

Which generated the following code:

```
<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Navibian Land Claim Portal - Omusati Region</title>




<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/leaflet.draw/1.0.4/leaflet.draw.css"
/>




<style>

    :root { --primary: #2c3e50; --secondary: #3498db; --success: #27ae60; --warning: #f39c12; --danger:
#e74c3c; }

    :root { --primary: #2c3e50; --secondary: #3498db; --success: #27ae60; --warning: #f39c12; --danger:
#e74c3c; --hearing: #9b59b6; }

    body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0;
background: #ecf0f1; color: #333; }




    /* Login Screen */

    #login-screen { display: flex; flex-direction: column; align-items: center; justify-content: center;
height: 100vh; }
```

```
.login-box { background: white; padding: 40px; border-radius: 8px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); width: 350px; text-align: center; }

.login-box input, .login-box select { width: 100%; padding: 10px; margin: 10px 0; border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; }

.btn { padding: 10px 15px; border: none; border-radius: 4px; background: var(--secondary); color: white; font-weight: bold; cursor: pointer; width: 100%; margin-top: 10px; }

.btn:hover { background: #2980b9; }


.login-hint { font-size: 11px; color: #888; margin-top: 8px; text-align: left; line-height: 1.5; }

.btn { padding: 10px 15px; border: none; border-radius: 4px; background: var(--secondary); color: white; font-weight: bold; cursor: pointer; width: 100%; margin-top: 10px; font-size: 13px; }

.btn:hover { opacity: 0.9; }


/* Main App Layout */

#app-screen { display: none; height: 100vh; flex-direction: column; }

header { background: var(--primary); color: white; padding: 15px 20px; display: flex; justify-content: space-between; align-items: center; }

header a { color: #a4cbed; text-decoration: none; font-size: 0.9em; }

header a:hover { text-decoration: underline; }


.main-container { display: flex; flex: 1; overflow: hidden; }

.sidebar { width: 350px; background: white; padding: 20px; overflow-y: auto; box-shadow: 2px 0 5px rgba(0,0,0,0.05); z-index: 2; }

#map { flex: 1; z-index: 1; }
```

```
/* Map wrapper: positions toast/dialog overlays relative to map area */
.map-wrapper { position: relative; flex: 1; overflow: hidden; }
#map { width: 100%; height: 100%; z-index: 1; }

/* ─── Map toast notification (replaces browser alert) ─── */
.map-toast {
  position: absolute; top: 16px; left: 50%; transform: translateX(-50%);
  z-index: 1100; padding: 14px 28px; border-radius: 8px;
  color: white; font-weight: 600; font-size: 14px; max-width: 500px; text-align: center;
  box-shadow: 0 4px 16px rgba(0,0,0,0.35); pointer-events: none;
  opacity: 0; transition: opacity 0.35s ease;
}
.map-toast.show  { opacity: 1; }
.map-toast.success { background: var(--success); }
.map-toast.error   { background: var(--danger); }
.map-toast.warning { background: var(--warning); }
.map-toast.info    { background: var(--secondary); }

/* ─── Map dialog overlay (replaces confirm / prompt) ─── */
.map-dialog-overlay {
  position: absolute; top: 0; left: 0; right: 0; bottom: 0;
  background: rgba(0,0,0,0.45); z-index: 1200;
  display: none; justify-content: center; align-items: center;
}
```

```
.map-dialog-overlay.show { display: flex; }
.map-dialog-box {
    background: white; padding: 24px; border-radius: 10px;
    max-width: 420px; width: 90%; box-shadow: 0 8px 32px rgba(0,0,0,0.35);
}
.map-dialog-box p { margin: 0 0 16px; font-size: 14px; line-height: 1.5; white-space: pre-line; }
.map-dialog-box textarea {
    width: 100%; padding: 10px; box-sizing: border-box;
    border: 1px solid #ccc; border-radius: 4px; margin-bottom: 14px;
    font-family: inherit; font-size: 13px; resize: vertical;
}
.dialog-btns { display: flex; gap: 10px; justify-content: flex-end; }
.dialog-btns .btn { width: auto; margin-top: 0; padding: 9px 20px; }
.btn-cancel { background: #7f8c8d !important; }


/* Claim Cards */
.claim-card { border: 1px solid #ddd; padding: 15px; margin-bottom: 15px; border-radius: 6px;
background: #f9f9f9; }
.claim-card { border: 1px solid #ddd; padding: 15px; margin-bottom: 15px; border-radius: 6px;
background: #f9f9f9; cursor: pointer; transition: box-shadow 0.2s; }
.claim-card:hover { box-shadow: 0 2px 8px rgba(0,0,0,0.15); }
.claim-card h3 { margin: 0 0 10px 0; font-size: 16px; }
.claim-card p { margin: 5px 0; font-size: 14px; }
.status-badge { display: inline-block; padding: 3px 8px; border-radius: 12px; font-size: 12px; color:
white; font-weight: bold;}
.status-pending { background: var(--warning); }
```

```
.status-approved { background: var(--success); }


/* Admin Popup Form */

.status-badge { display: inline-block; padding: 3px 8px; border-radius: 12px; font-size: 12px; color: white; font-weight: bold; }

.status-pending   { background: var(--warning); }

.status-approved  { background: var(--success); }

.status-rejected  { background: var(--danger); }

.status-hearing   { background: var(--hearing); }

.rejection-reason { color: #e74c3c; font-size: 12px; margin-top: 4px; padding: 6px; background: #fdecea; border-radius: 4px; }


/* Admin Popup Form (inside Leaflet map popups) */

.admin-action-form { margin-top: 10px; border-top: 1px solid #ccc; padding-top: 10px; }

.admin-action-form input { width: 100%; padding: 8px; margin-bottom: 10px; box-sizing: border-box; }

.admin-action-form input { width: 100%; padding: 8px; margin-bottom: 10px; box-sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }

.btn-approve { background: var(--success); width: 48%; }

.btn-reject { background: var(--danger); width: 48%; }

.btn-logout { background: #7f8c8d; width: auto; padding: 8px 15px;}

.btn-reject  { background: var(--danger);  width: 48%; }

.btn-logout  { background: #7f8c8d; width: auto; padding: 8px 15px; }

.btn-remove  { background: var(--danger); font-size: 12px; padding: 5px 10px; width: auto; margin-top: 8px; }

.btn-dismiss { background: #7f8c8d; font-size: 12px; padding: 5px 10px; width: auto; margin-top: 8px; }
```

```
    </style>
</head>
<body>


    <!-- ═══════════════ LOGIN SCREEN ═══════════════ -->
    <div id="login-screen">
        <div class="login-box">
            <h2>Portal Login</h2>
@@ -55,12 +101,18 @@
                <option value="ta">Traditional Authority (TA) Admin</option>
                <option value="clb">Communal Land Board (CLB) Admin</option>
            </select>
            <input type="text" id="email-input" placeholder="Email / Username"
value="citizen@omusati.na">
            <input type="password" id="password-input" placeholder="Password" value="password123">
            <input type="text"     id="email-input"    placeholder="Email / Username">
            <input type="password" id="password-input" placeholder="Password">
            <button class="btn" onclick="login()">Access System</button>
            <div class="login-hint">
                <b>Citizen:</b> any email &middot; password: <code>citizen123</code><br>
                <b>TA Admin:</b> ta@omusati.na &middot; password: <code>ta_pass_789</code><br>
                <b>CLB Admin:</b> clb@omusati.na &middot; password: <code>clb_pass_456</code>
            </div>
        </div>
    </div>
```

```
<!-- ═══════════════ APP SCREEN ═══════════════ -->
<div id="app-screen">
  <header>
    <div>
@@ -72,218 +124,612 @@
      <button class="btn btn-logout" style="margin-left: 15px;" onclick="logout()">Logout</button>
    </div>
  </header>



  <div class="main-container">
    <div class="sidebar">
      <h3 id="sidebar-title">My Claims</h3>
      <div id="claims-list">
        </div>
      <div id="claims-list"></div>
    </div>

    <!-- Map wrapper: toast and dialog are positioned inside here -->
    <div class="map-wrapper">
      <div id="map"></div>
      <div id="map-toast" class="map-toast"></div>
      <div id="map-dialog-overlay" class="map-dialog-overlay">
```

```
            <div class="map-dialog-box" id="map-dialog-box"></div>
          </div>
        </div>

        <div id="map"></div>
    </div>
</div>

<!-- ═══════════════ LIBRARIES ═══════════════ -->
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/leaflet.draw/1.0.4/leaflet.draw.js"></script>
<script src="https://cdn.jsdelivr.net/npm/pouchdb@8.0.1/dist/pouchdb.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@turf/turf@6/turf.min.js"></script>

<script>
    // --- SYSTEM STATE & DB ---
    const db = new PouchDB('omusati_land_registry');
    let currentUser = null; // { role, email }

    let map, drawControl;
    const allMapLayers = new L.FeatureGroup();
    let savedFeatures = []; // Pure math geometry for Turf.js collision detection

    // Simulated Protected Water (Olushandja Dam area in Omusati)
```

```
const waterFeature = turf.polygon([[
    [14.65, -17.40], [14.70, -17.40], [14.70, -17.48], [14.65, -17.48], [14.65, -17.40]
]]);

// --- 1. AUTHENTICATION ---
function login() {
    const role = document.getElementById('role-select').value;
    const email = document.getElementById('email-input').value;

    if (!email) return alert("Please enter an email/username.");

    currentUser = { role, email };

    document.getElementById('login-screen').style.display = 'none';
    document.getElementById('app-screen').style.display = 'flex';

    // Adjust UI based on role
    document.getElementById('header-title').innerText = role === 'citizen' ? "MunyaMap Land Portal" : (role === 'ta' ? "MunyaMap: Traditional Authority Admin Portal" : "Munya Map: Communal Land Board Admin Portal");
    document.getElementById('sidebar-title').innerText = role === 'citizen' ? "My Pending Claims" : "All System Claims";
    if (role === 'citizen') document.getElementById('user-info').innerText = `Logged in as: ${email}`;

    initMap();
    loadClaims();
```

```
    }

// ═══════════════════════════════════════════════════════════════════════════
// SYSTEM STATE & DATABASE
// ═══════════════════════════════════════════════════════════════════════════

const db = new PouchDB('omusati_land_registry');

let currentUser = null;          // { role, email }


let map, drawControl;

const allMapLayers = new L.FeatureGroup();

let savedFeatures  = [];         // Active claim geometries for collision detection

let claimLayerMap  = {};         // claim._id → Leaflet layer (click-to-navigate)

let waterPolygons  = [];         // Turf polygon features for water bodies

let waterMapLayer  = null;       // Leaflet layer group for rendered water overlays


// Debounce timer for loadClaims – prevents double-rendering

let _loadClaimsTimer = null;


// ── Credentials ──────────────────────────────────────────────────────────

const CREDENTIALS = {

    citizen: { password: 'citizen123' },

    ta:      { email: 'ta@omusati.na',  password: 'ta_pass_789'  },

    clb:     { email: 'clb@omusati.na', password: 'clb_pass_456' }
```

```
};

// ── Geographic constants ──────────────────────────────────────────────

// Namibia bounding box (simplified)
const namibiaBounds = turf.polygon([[
    [11.7, -17.0], [25.3, -17.0], [25.3, -29.0], [11.7, -29.0], [11.7, -17.0]
]]);

    function logout() {
        currentUser = null;
        document.getElementById('app-screen').style.display = 'none';
        document.getElementById('login-screen').style.display = 'flex';
        if(map) { map.remove(); map = null; }
// Omusati service-region border (approximate) — drawn on map as dashed line
const omusatiCoords = [
    [-17.35, 14.00], [-17.35, 15.50], [-17.55, 15.70],
    [-18.00, 15.70], [-18.45, 15.40], [-18.50, 15.00],
    [-18.30, 14.10], [-17.90, 13.80], [-17.50, 13.90], [-17.35, 14.00]
];

// Hardcoded fallback water polygon: Olushandja Dam [lon, lat]
const olushandjaDam = turf.polygon([[
    [14.65, -17.40], [14.70, -17.40], [14.70, -17.48], [14.65, -17.48], [14.65, -17.40]
]]);
```

```
waterPolygons.push(olushandjaDam);


//
═══════════════════════════════════════════════════════════════════════════════
═══════════════════

//  CUSTOM MAP NOTIFICATIONS (replace browser alert / confirm / prompt)

//
═══════════════════════════════════════════════════════════════════════════════
═══════════════════


/** Toast notification that appears over the map */

function notify(message, type = 'info', duration = 4000) {

    const el = document.getElementById('map-toast');

    el.textContent = message;

    el.className = 'map-toast ' + type;          // set colour class

    void el.offsetWidth;                         // force reflow

    el.classList.add('show');

    clearTimeout(el._t);

    el._t = setTimeout(() => el.classList.remove('show'), duration);

}


/** Confirm dialog over the map – returns Promise<boolean> */

function mapConfirm(message) {

    return new Promise(resolve => {

        const overlay = document.getElementById('map-dialog-overlay');

        const box     = document.getElementById('map-dialog-box');
```

```
        box.innerHTML = `
            <p>${message}</p>
            <div class="dialog-btns">
                <button class="btn btn-cancel" id="dlg-no">Cancel</button>
                <button class="btn"           id="dlg-yes">Confirm</button>
            </div>`;
        overlay.classList.add('show');
        const close = val => { overlay.classList.remove('show'); resolve(val); };
        document.getElementById('dlg-yes').onclick = () => close(true);
        document.getElementById('dlg-no').onclick  = () => close(false);
    });
}

/** Prompt dialog over the map – returns Promise<string|null> */
function mapPrompt(message) {
    return new Promise(resolve => {
        const overlay = document.getElementById('map-dialog-overlay');
        const box     = document.getElementById('map-dialog-box');
        box.innerHTML = `
            <p>${message}</p>
            <textarea id="dlg-input" rows="3" placeholder="Enter your response here..."></textarea>
            <div class="dialog-btns">
                <button class="btn btn-cancel" id="dlg-cancel">Cancel</button>
                <button class="btn"           id="dlg-ok">Submit</button>
```

```
            </div>`;
        overlay.classList.add('show');
        const close = val => { overlay.classList.remove('show'); resolve(val); };
        document.getElementById('dlg-ok').onclick     = () =>
close(document.getElementById('dlg-input').value);
        document.getElementById('dlg-cancel').onclick = () => close(null);
    });
}

// ═══════════════════════════════════════════════════════════════════════════
═══════════════════

// 1. AUTHENTICATION
// ═══════════════════════════════════════════════════════════════════════════
═══════════════════

function login() {
    const role     = document.getElementById('role-select').value;
    const email    = document.getElementById('email-input').value.trim();
    const password = document.getElementById('password-input').value;

    if (!email)    return alert("Please enter an email/username.");
    if (!password) return alert("Please enter a password.");

    // Validate credentials per role
    if (role === 'citizen') {
        if (password !== CREDENTIALS.citizen.password)
```

```
        return alert("Invalid password. Access denied.");
    } else if (role === 'ta') {
        if (email !== CREDENTIALS.ta.email || password !== CREDENTIALS.ta.password)
            return alert("Invalid TA Admin credentials. Access denied.");
    } else if (role === 'clb') {
        if (email !== CREDENTIALS.clb.email || password !== CREDENTIALS.clb.password)
            return alert("Invalid CLB Admin credentials. Access denied.");
    }

// --- 2. MAP INITIALIZATION ---
function initMap() {
    // Centered on Omusati Region
    map = L.map('map').setView([-17.45, 14.68], 12);
    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', { maxZoom: 18 }).addTo(map);
    map.addLayer(allMapLayers);

    // Setup Drawing Tools ONLY for Citizens
    if (currentUser.role === 'citizen') {
        drawControl = new L.Control.Draw({
            edit: false,
            draw: { marker: false, circle: false, circlemarker: false, polyline: false, polygon: true }
        });
        map.addControl(drawControl);
```

```
        // Handle Citizen Drawing
        map.on(L.Draw.Event.CREATED, async function (event) {
            const newGeoJSON = event.layer.toGeoJSON();

            // Validation 1: Size Limit (< 50 Hectares)
            const areaSqMeters = turf.area(newGeoJSON);
            const areaHectares = areaSqMeters / 10000;
            if (areaHectares > 50) {
                return alert(`❌ Claim Denied: Your plot is ${areaHectares.toFixed(2)} ha. Maximum
allowed by CLRA is 50 ha.`);
            }

            // Validation 3: Existing Claim Overlap
            let collision = savedFeatures.some(f => turf.booleanIntersects(newGeoJSON, f));
            if (collision) {
                return alert("❌ Claim Denied: Boundary overlaps with an existing pending or approved
claim.");
            }

            // If valid, save to database
            createNewClaim(newGeoJSON.geometry, areaHectares);
        });
    currentUser = { role, email };

    document.getElementById('login-screen').style.display = 'none';
```

```
    document.getElementById('app-screen').style.display   = 'flex';


    document.getElementById('header-title').innerText =
        role === 'citizen' ? "MunyaMap Land Portal" :
        role === 'ta'      ? "MunyaMap: Traditional Authority Admin Portal" :
                             "MunyaMap: Communal Land Board Admin Portal";
    document.getElementById('sidebar-title').innerText =
        role === 'citizen' ? "My Claims" : "All System Claims";
    document.getElementById('user-info').innerText = `Logged in as: ${email}`;


    initMap();
    loadClaims();   // initial load (debounced listener handles subsequent updates)
}


function logout() {
    currentUser   = null;
    claimLayerMap = {};
    document.getElementById('app-screen').style.display   = 'none';
    document.getElementById('login-screen').style.display = 'flex';
    if (map) { map.remove(); map = null; }
}


//
================================================================================
=================
```

```
// 2. MAP INITIALIZATION
//
═══════════════════════════════════════════════════════════════════════════════
═══════════════════

function initMap() {
  map = L.map('map', { zoomControl: true }).setView([-17.45, 14.68], 12);


  // ── Base layer: standard OpenStreetMap (shows water in blue) ──
  const osmBase = L.tileLayer(
    'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',
    {
      maxZoom: 19,
      attribution: '&copy; OpenStreetMap contributors',
      // 1×1 transparent PNG as error-tile – prevents "map data no longer appears"
      errorTileUrl:
'
0lEQVR42mNkYAAAAAYAAjCB0C8AAAAASUVORK5CYII='
    }
  );


  // ── Overlay: ESRI satellite imagery at 65 % opacity ──
  const satOverlay = L.tileLayer(
    'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}',
    {
      maxZoom: 19,
      maxNativeZoom: 17,
```

```
            opacity: 0.65,

            attribution: '&copy; Esri, Maxar, Earthstar Geographics',

            errorTileUrl:
'
0lEQVR42mNkYAAAAAYAAjCB0C8AAAAASUVORK5CYII='

        }

    );


    osmBase.addTo(map);

    satOverlay.addTo(map);


    // Layer control so user can toggle satellite overlay

    L.control.layers(null, { 'Satellite Overlay': satOverlay }).addTo(map);


    map.addLayer(allMapLayers);


    // ── Draw Omusati service-region border ──

    L.polygon(omusatiCoords, {

        color: '#e67e22', weight: 2.5, dashArray: '10,6',

        fillOpacity: 0, interactive: false

    }).addTo(map).bindTooltip('Omusati Service Region', { permanent: false, direction: 'center' });


    // ── Render hardcoded water overlay ──

    renderWaterOverlay();
```

```
// ── Fetch live water bodies from OpenStreetMap (Overpass API) ──
fetchWaterBodies();

// Fix Leaflet rendering after container becomes visible
setTimeout(() => map.invalidateSize(), 200);

// ── Drawing tools (citizens only) ──
if (currentUser.role === 'citizen') {
  drawControl = new L.Control.Draw({
    edit: false,
    draw: {
      marker:       false,
      circle:       false,
      circlemarker: false,
      polyline:     false,
      rectangle:    false,   // prevents unwanted blue square
      polygon:      true
    }
  });
  map.addControl(drawControl);

  map.on(L.Draw.Event.CREATED, async function (event) {
    const newGeoJSON    = event.layer.toGeoJSON();
    const areaHectares  = turf.area(newGeoJSON) / 10000;
```

```
// Validation 1 – Size limit (CLRA max 50 ha)
if (areaHectares > 50) {
    return notify(`Claim Denied: Your plot is ${areaHectares.toFixed(2)} ha. Maximum allowed by CLRA is 50 ha.`, 'error', 5000);
}

// Validation 2 – Must be inside Namibia
if (!turf.booleanContains(namibiaBounds, turf.feature(newGeoJSON.geometry))) {
    return notify("Claim Denied: Your claim must be located within Namibia.", 'error', 5000);
}

// Validation 3 – No water bodies
const hitsWater = waterPolygons.some(wp => {
    try { return turf.booleanIntersects(newGeoJSON, wp); } catch(e) { return false; }
});
if (hitsWater) {
    return notify("Claim Denied: Your claim overlaps with a body of water.", 'error', 5000);
}

// Validation 4 – Existing claim overlap → offer hearing request
const overlaps = savedFeatures.some(f => turf.booleanIntersects(newGeoJSON, f));
if (overlaps) {
    const wantHearing = await mapConfirm(
        "Your claim overlaps with an existing claim.\n\n" +
```

```
            "You cannot submit a standard claim here.\n\n" +
            "Would you like to submit a formal hearing request instead?"
        );
        if (wantHearing) await createHearingRequest(newGeoJSON.geometry, areaHectares);
        return;
    }

    // All validations passed
    await createNewClaim(newGeoJSON.geometry, areaHectares);
  });
}
}

// --- 3. CLAIM CREATION & SYNC ---
async function createNewClaim(geometry, hectares) {
  // Generate the secure key for the TA/CLB to verify
  const cryptoKey = Math.random().toString(36).substring(2, 10).toUpperCase();

  const claim = {
    _id: 'claim_' + new Date().toISOString(),
    claimerEmail: currentUser.email,
    date: new Date().toLocaleDateString(),
    area: hectares.toFixed(2),
    boundary: geometry,
```

```
            cryptoKey: cryptoKey, // Passed to DB for admin verification
            status: 'pending_ta', // Workflow: pending_ta -> pending_clb -> approved
            type: 'land_claim'
        };

        await db.put(claim);
        alert("✅ Claim Submitted! Awaiting Traditional Authority (TA) approval.");
        loadClaims();
    //
═══════════════════════════════════════════════════════════════════════════════════════
    //  WATER BODY MANAGEMENT
    //
═══════════════════════════════════════════════════════════════════════════════════════

    /** Render all known water polygons as blue overlays on the map */
    function renderWaterOverlay() {
        if (waterMapLayer) map.removeLayer(waterMapLayer);
        waterMapLayer = L.layerGroup();
        waterPolygons.forEach(wp => {
            try {
                L.geoJSON(wp, {
                    style: { color: '#0077be', weight: 1, fillOpacity: 0.25, fillColor: '#2196F3' },
                    interactive: false
                }).addTo(waterMapLayer);
```

```
        } catch(e) { /* skip bad geometries */ }
    });
    waterMapLayer.addTo(map);
}


/** Fetch water bodies from OpenStreetMap via Overpass API for the Omusati region */
async function fetchWaterBodies() {
    try {
        // south, west, north, east  (Omusati region bbox)
        const bbox  = '-18.6,13.7,-17.2,15.8';
        const query =
`[out:json][timeout:30];(way["natural"="water"](${bbox});relation["natural"="water"](${bbox});way["w
aterway"="riverbank"](${bbox}););out geom;`;
        const resp  = await fetch('https://overpass-api.de/api/interpreter?data=' +
encodeURIComponent(query));
        if (!resp.ok) throw new Error('Overpass API error');
        const data = await resp.json();


        let added = 0;
        data.elements.forEach(el => {
            // Process ways directly
            if (el.type === 'way' && el.geometry && el.geometry.length >= 4) {
                const poly = wayGeomToTurf(el.geometry);
                if (poly) { waterPolygons.push(poly); added++; }
            }
            // Process relation outer members
```

```
        if (el.type === 'relation' && el.members) {
          el.members.forEach(m => {
            if (m.role === 'outer' && m.geometry && m.geometry.length >= 4) {
              const poly = wayGeomToTurf(m.geometry);
              if (poly) { waterPolygons.push(poly); added++; }
            }
          });
        }
      });

      if (added > 0) renderWaterOverlay();
      console.log(`Loaded ${added} water body polygons from OpenStreetMap.`);
    } catch (e) {
      console.warn('Water body fetch failed — using fallback polygons.', e);
    }
}

/** Convert an Overpass way geometry array [{lat,lon},...] → Turf polygon (or null) */
function wayGeomToTurf(geom) {
    try {
      const coords = geom.map(g => [g.lon, g.lat]);
      // Close the ring if needed
      const first = coords[0], last = coords[coords.length - 1];
      if (first[0] !== last[0] || first[1] !== last[1]) coords.push([first[0], first[1]]);
```

```
        if (coords.length < 4) return null;
        return turf.polygon([coords]);
    } catch(e) { return null; }
}

    // --- 4. RENDER CLAIMS (MAP & LIST) ---
    async function loadClaims() {
        allMapLayers.clearLayers();
        savedFeatures = [];
        const claimsListDiv = document.getElementById('claims-list');
        claimsListDiv.innerHTML = '';
//
================================================================================================================
//  3. CLAIM CREATION
//
================================================================================================================
async function createNewClaim(geometry, hectares) {
    const taCryptoKey  = Math.random().toString(36).substring(2, 10).toUpperCase();
    const clbCryptoKey = Math.random().toString(36).substring(2, 10).toUpperCase();

        const result = await db.allDocs({include_docs: true});

        result.rows.forEach(row => {
            const claim = row.doc;
```

```
        if(claim.type !== 'land_claim') return;
    const claim = {
      _id:         'claim_' + new Date().toISOString() + '_' + Math.random().toString(36).substring(2, 6),
      claimerEmail: currentUser.email,
      date:        new Date().toLocaleDateString(),
      area:        hectares.toFixed(2),
      boundary:    geometry,
      taCryptoKey,
      clbCryptoKey,
      status:      'pending_ta',
      type:        'land_claim'
    };

        // Filter citizen view to only show their claims
        if (currentUser.role === 'citizen' && claim.claimerEmail !== currentUser.email) return;
    await db.put(claim);
    notify("Claim Submitted! Awaiting Traditional Authority (TA) approval.", 'success');
    // loadClaims() fires via the debounced db.changes listener — no explicit call here
}

        // 1. Add to Math Ledger for collision detection
async function createHearingRequest(geometry, hectares) {
    const claim = {
      _id:         'claim_' + new Date().toISOString() + '_' + Math.random().toString(36).substring(2, 6),
```

```
        claimerEmail: currentUser.email,
        date:         new Date().toLocaleDateString(),
        area:         hectares.toFixed(2),
        boundary:     geometry,
        taCryptoKey:  '',
        clbCryptoKey: '',
        status:       'hearing_requested',
        type:         'land_claim'
      };
      await db.put(claim);
      notify("Hearing request submitted. You will be contacted with hearing details.", 'info');
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // 4. RENDER CLAIMS  (MAP & SIDEBAR)
    //    – debounced so rapid db.changes events don't cause duplicates
    // ═══════════════════════════════════════════════════════════════════════════════

    function scheduleLoadClaims() {
      clearTimeout(_loadClaimsTimer);
      _loadClaimsTimer = setTimeout(loadClaims, 120);
    }
```

```
async function loadClaims() {
    if (!map || !currentUser) return;

    allMapLayers.clearLayers();
    savedFeatures = [];
    claimLayerMap = {};

    const result = await db.allDocs({ include_docs: true });

    let sidebarHTML = '';     // build sidebar HTML in one pass, set once at the end

    result.rows.forEach(row => {
        const claim = row.doc;
        if (claim.type !== 'land_claim') return;

        const isOwnClaim = claim.claimerEmail === currentUser.email;
        const isAdmin    = currentUser.role === 'ta' || currentUser.role === 'clb';

        // ── Rejected-claim visibility rules ──
        // Rejected claims ONLY visible to the citizen who owns them.
        // They do NOT appear on TA or CLB portals at all.
        if (claim.status === 'rejected') {
            if (!isOwnClaim || isAdmin) return;          // skip entirely for admins & other citizens
```

```
}

// Collision detection: only active (non-rejected) claims
if (claim.status !== 'rejected') {
    savedFeatures.push(turf.feature(claim.boundary));
}

    // 2. Add to Map
    let color = claim.status === 'approved' ? '#27ae60' : '#f39c12';
    const layer = L.geoJSON(claim.boundary, { style: { color: color, weight: 2, fillOpacity: 0.4 } });

    // Construct Popup
    let popupContent = `
        <b>Claim ID:</b> ${claim._id.split('_')[1]}<br>
        <b>Claimer:</b> ${claim.claimerEmail}<br>
        <b>Area:</b> ${claim.area} hectares<br>
        <b>Status:</b> ${claim.status.replace('_', ' ').toUpperCase()}<br>
    `;

    // Admin Verification Form in Popup
    if ((currentUser.role === 'ta' && claim.status === 'pending_ta') ||
        (currentUser.role === 'clb' && claim.status === 'pending_clb')) {
        popupContent += `
            <div class="admin-action-form">
```

```
                <input type="text" id="key-${claim._id}" placeholder="Enter Crypto Key to Verify">

                <button class="btn btn-approve" onclick="verifyAndApprove('${claim._id}',
'${currentUser.role}')">Approve</button>

                <button class="btn btn-reject" onclick="rejectClaim('${claim._id}')">Deny</button>

            </div>

        `;

    }

    // Sidebar visibility: citizens see only their own; admins see all (except rejected, filtered above)

    const showInSidebar = isAdmin || isOwnClaim;



    // ── Colour coding ──

    let color;

    if (currentUser.role === 'citizen' && !isOwnClaim) {

        color = '#3498db';          // other citizens' claims → blue

    } else if (claim.status === 'approved')          { color = '#27ae60'; }

      else if (claim.status === 'rejected')          { color = '#e74c3c'; }

      else if (claim.status === 'hearing_requested') { color = '#9b59b6'; }

      else                                           { color = '#f39c12'; }


        layer.bindPopup(popupContent);

        allMapLayers.addLayer(layer);


        // 3. Add to Sidebar List

        let badgeClass = claim.status === 'approved' ? 'status-approved' : 'status-pending';
```

```
            let adminKeyNote = (currentUser.role !== 'citizen') ? `<br><small style="color:red;">Secure
Key: ${claim.cryptoKey}</small>` : '';



            claimsListDiv.innerHTML += `

                <div class="claim-card">

                    <h3>Claim: ${claim._id.split('_')[1].substring(0, 10)}...</h3>

                    <p><b>Date:</b> ${claim.date}</p>

                    <p><b>Size:</b> ${claim.area} ha</p>

                    <p><span class="status-badge ${badgeClass}">${claim.status.replace('_', '
').toUpperCase()}</span></p>

                    ${adminKeyNote}

                </div>

            `;

        // ── Add polygon to map ──

        const layer = L.geoJSON(claim.boundary, {

            style: { color, weight: 2, fillOpacity: 0.4 }

        });

    }


    // --- 5. ADMIN VERIFICATION LOGIC ---

    async function verifyAndApprove(claimId, role) {

        const enteredKey = document.getElementById(`key-${claimId}`).value;

        const claim = await db.get(claimId);

        let popupContent = `

            <b>Claim ID:</b> ${shortId(claim._id)}<br>
```

```
        <b>Claimer:</b> ${claim.claimerEmail}<br>
        <b>Area:</b> ${claim.area} ha<br>
        <b>Status:</b> ${claim.status.replace(/_/g, ' ').toUpperCase()}<br>
    `;
    if (claim.status === 'rejected' && claim.rejectionReason) {
        popupContent += `<b style="color:#e74c3c;">Reason:</b> ${claim.rejectionReason}<br>`;
    }

    if (enteredKey !== claim.cryptoKey) {
        return alert("❌ Invalid Crypto Key. Verification failed.");
    // Admin verification form (in Leaflet popup on the polygon)
    if ((currentUser.role === 'ta'  && claim.status === 'pending_ta') ||
        (currentUser.role === 'clb' && claim.status === 'pending_clb')) {
        popupContent += `
            <div class="admin-action-form">
                <input type="text" id="key-${claim._id}" placeholder="Enter Crypto Key to Verify">
                <button class="btn btn-approve"
onclick="verifyAndApprove('${claim._id}','${currentUser.role}')">Approve</button>
                <button class="btn btn-reject"
onclick="rejectClaim('${claim._id}','${currentUser.role}')">Deny</button>
            </div>`;
    }

    // Advance workflow
    if (role === 'ta') claim.status = 'pending_clb';
```

```
if (role === 'clb') claim.status = 'approved';


await db.put(claim);
map.closePopup();
alert(`✅ Claim ${role.toUpperCase()} Approved.`);
loadClaims();
layer.bindPopup(popupContent);
allMapLayers.addLayer(layer);
claimLayerMap[claim._id] = layer;


// ── Sidebar card ──
if (!showInSidebar) return;


const badgeClass =
    claim.status === 'approved'          ? 'status-approved' :
    claim.status === 'rejected'          ? 'status-rejected' :
    claim.status === 'hearing_requested' ? 'status-hearing'  :
    'status-pending';


// Show the relevant crypto key to the relevant admin
let adminKeyNote = '';
if (currentUser.role === 'ta'  && claim.status === 'pending_ta'  && claim.taCryptoKey)
    adminKeyNote = `<p><small style="color:red;"><b>TA Verify Key:</b>
${claim.taCryptoKey}</small></p>`;
else if (currentUser.role === 'clb' && claim.status === 'pending_clb' && claim.clbCryptoKey)
```

```
            adminKeyNote = `<p><small style="color:red;"><b>CLB Verify Key:</b>
${claim.clbCryptoKey}</small></p>`;


        // Rejection reason notification for citizen

        let rejectionNote = '';

        if (claim.status === 'rejected' && claim.rejectionReason)

            rejectionNote = `<p class="rejection-reason"><b>Rejected:</b>
${claim.rejectionReason}</p>`;


        // Action buttons

        let actionBtns = '';

        if (currentUser.role === 'citizen' && isOwnClaim) {

            if (claim.status === 'pending_ta') {

                actionBtns = `<button class="btn btn-remove"
onclick="removeCitizenClaim('${claim._id}',event)">Remove Claim</button>`;

            } else if (claim.status === 'rejected') {

                // Citizen can dismiss (permanently delete) a rejected notification

                actionBtns = `<button class="btn btn-dismiss"
onclick="dismissRejected('${claim._id}',event)">OK &ndash; Dismiss</button>`;

            }

        }


        sidebarHTML += `

            <div class="claim-card" onclick="navigateToClaim('${claim._id}')">

                <h3>Claim: ${shortId(claim._id)}</h3>

                <p><b>Date:</b> ${claim.date}</p>
```

```
                <p><b>Size:</b> ${claim.area} ha</p>

                <p><span class="status-badge ${badgeClass}">${claim.status.replace(/_/g, '
').toUpperCase()}</span></p>

                ${rejectionNote}

                ${adminKeyNote}

                ${actionBtns}

            </div>`;

        });


        document.getElementById('claims-list').innerHTML = sidebarHTML;

    }


    /** Shorten a claim _id for display */

    function shortId(id) {

        const parts = id.split('_');

        return parts.length > 1 ? parts[1].substring(0, 10) + '...' : id.substring(0, 14) + '...';

    }


    // ── Sidebar click → pan map to claim ──

    function navigateToClaim(claimId) {

        const layer = claimLayerMap[claimId];

        if (layer) {

            map.fitBounds(layer.getBounds(), { padding: [50, 50] });

            layer.eachLayer(l => { if (l.openPopup) l.openPopup(); });

        }
```

```
}

//
═══════════════════════════════════════════════════════════════════════════
═══════════════

// 5. CITIZEN ACTIONS

//
═══════════════════════════════════════════════════════════════════════════
═══════════════

/** Citizen removes their own pending claim */

async function removeCitizenClaim(claimId, event) {

    event.stopPropagation();

    const yes = await mapConfirm("Are you sure you want to remove this claim?\nThis cannot be
undone.");

    if (!yes) return;

    const claim = await db.get(claimId);

    await db.remove(claim);

    notify("Your claim has been removed.", 'success');

}


    async function rejectClaim(claimId) {

        if(!confirm("Are you sure you want to completely deny and remove this claim?")) return;

        const claim = await db.get(claimId);

        await db.remove(claim); // If rejected, it is removed from the map

        map.closePopup();

        alert("❌ Claim has been denied and removed from the ledger.");
```

```
        loadClaims();
/** Citizen dismisses a rejected claim notification (permanently deletes it) */
async function dismissRejected(claimId, event) {
    event.stopPropagation();
    const claim = await db.get(claimId);
    await db.remove(claim);
    notify("Rejection dismissed.", 'info', 2500);
}

//
================================================================================================================
//  6. ADMIN VERIFICATION LOGIC
//
================================================================================================================

async function verifyAndApprove(claimId, role) {
    const enteredKey = (document.getElementById(`key-${claimId}`) || {}).value || '';
    const claim      = await db.get(claimId);
    const expectedKey = role === 'ta' ? claim.taCryptoKey : claim.clbCryptoKey;

    if (!enteredKey || enteredKey !== expectedKey) {
        return notify("Invalid Crypto Key. Verification failed.", 'error');
    }
```

```
    // Live Sync Listener (Updates UI instantly if another device makes a change)
    db.changes({ since: 'now', live: true, include_docs: true }).on('change', () => {
        if(currentUser) loadClaims();
    });
    if (role === 'ta')  claim.status = 'pending_clb';
    if (role === 'clb') claim.status = 'approved';

    await db.put(claim);
    map.closePopup();
    notify(`Claim ${role.toUpperCase()} Approved.`, 'success');
    // loadClaims fires via debounced sync listener
}

async function rejectClaim(claimId, role) {
    const keyInput   = document.getElementById(`key-${claimId}`);
    const enteredKey = keyInput ? keyInput.value.trim() : '';
    const claim      = await db.get(claimId);
    const expectedKey = role === 'ta' ? claim.taCryptoKey : claim.clbCryptoKey;

    if (!enteredKey || enteredKey !== expectedKey) {
        return notify("You must enter a valid Crypto Key before denying a claim.", 'error');
    }

    const sure = await mapConfirm("Are you sure you want to deny this claim?");
```

```
    if (!sure) return;

    // CLB must provide a reason
    let rejectionReason = '';
    if (role === 'clb') {
        rejectionReason = await mapPrompt(
            "CLB Requirement: Please enter the reason for rejection.\n" +
            "This will be sent to the citizen as a notification."
        );
        if (!rejectionReason || !rejectionReason.trim()) {
            return notify("A rejection reason is required from the CLB.", 'error');
        }
    }

    claim.status          = 'rejected';
    claim.rejectionReason = (rejectionReason || `Denied by ${role.toUpperCase()}`).trim();
    await db.put(claim);
    map.closePopup();
    notify("Claim denied. The citizen will be notified.", 'warning');
}

//
================================================================================================================

//  7. LIVE SYNC LISTENER (debounced)
```

```
  //
═══════════════════════════════════════════════════════════════════════════════
═══════════════════

  db.changes({ since: 'now', live: true, include_docs: true }).on('change', () => {

    if (currentUser) scheduleLoadClaims();

  });



  </script>

</body>
```

Review the specification above. Analyze the current codebase and fix these issues one by one."

Slide Generation (Manus AI)

Prompt

- *"Read this and make slides accordingly. This is used for the results shown in hackathon."*
- Attached manuscript

Generated Slides